

通信プロトコルにおけるサービス不能攻撃耐性解析のための型付き π 計算

富岡大悟 池田立野 西崎真也

通信プロトコルの安全性, 特に認証プロトコルにおける認証の正当性に関する研究は, Abadi や Gordon による spi 計算 (secure pi-calculus) などをはじめとして, 近年さかんに行なわれている. プロトコルにおける安全性は, 認証の正当性や機密性などの他に, 最近では, サービス不能攻撃 (Denial-of-Service attack) 耐性が重要視される. もっとも典型的な攻撃例としては, TCP の 3 ウェイハンドシェイクにおける SYN あふれ攻撃 (SYN-flooding attack) が知られている. プロトコルのサービス不能攻撃耐性を形式的に扱う枠組みとしては, メドーズらにより提唱された, コスト情報を付記したアリス・ボブ記法があった. この他に, 最近, 富岡らにより提唱された spice 計算 [13] がある. spice 計算は, spi 計算を拡張したもので, プロセスの計算における計算コストが, サーバーやクライアントの計算機において, どのように費されるかを明示的に表現できるように, システムにおける計算機構成を型として形式化した. そして, 書き換えスタイルの操作的意味論があたえられており, プロセスに対する型付けの情報を利用することにより, 計算の進行における計算コストを区別するようになっている. 記憶コストは, サービス不能攻撃耐性を測る場合, 各種の計算コストのうちで最も重要となるのだが, spice 計算では, 記憶領域の解放を明示的に行なうようにした. 本研究では, 従来の spice 計算における型体系と操作的意味論を, 計算機ごとの記憶コストの見積りに併せて, 記憶領域の解放に関する正当性を保証するように, 改良した. また, SYN あふれ攻撃とその防御策である SYN クッキーが形式化できることを適用例として紹介する.

Security on communication protocols, especially authentication protocols, has been studied by many researchers, for example, the secure pi-calculus by Abadi and Gordon. Resistance against the denial-of-service attack is one of the important issues of secure protocols as secrecy and authenticity. A typical example of the denial-of-service attack is TCP SYN-flooding attack. Meadows proposed an Alice-and-Bob notation annotated with cost description as a formal framework for analyzing resistance against denial-of-service attacks. In this paper, we proposed the spice-calculus, which is an extended secure pi-calculus by adding explicit description on computation costs. We give operational semantics and a type system to the spice-calculus; the typing represents configuration of network system. We then investigate theoretical properties on the type system, e.g. subject reduction. We present formalization of TCP SYN flooding attack and its protection method, SYN-cookie, on the typed spice-calculus as examples.

1 はじめに

1.1 サービス不能攻撃

サービス不能攻撃 (Denial-of-Service attack, DoS-attack) は, ソフトウェアにおける安全性をおびやかす典型的なものであり, 正規のユーザーが計算機や

ネットワークの使用を不可能におこむような攻撃である. 典型的な DoS 攻撃の例としては, TCP における SYN あふれ攻撃 (SYN flooding attack) [12] があげられる.

サービス不能攻撃の例: SYN あふれ攻撃

TCP の通信においては, 3 ウェイハンドシェイクと呼ばれるプロトコルに基づいて接続が確立される. 接続元を S とし, 接続先を D とする.

1. まず, S は SYN と呼ばれるパケットを D に送る. SYN パケットには, 初期シーケンス番号 x が含まれる. シーケンス番号 (sequence number, SEQ) とは, 一連のデータ送信において, どこま

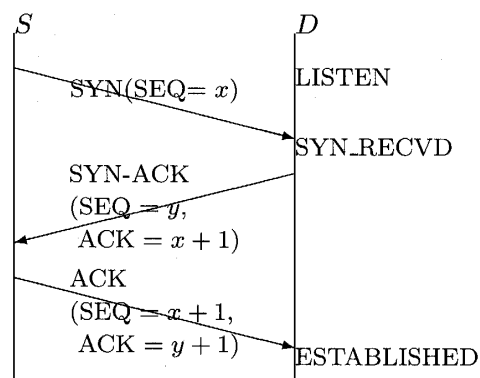
Typed pi-calculus for analysis of resistance against denial-of-service attacks in communication protocols.

Daigo Tomioka, Tatsuya Ikeda, Shin-ya Nishizaki, 東京工業大学大学院情報理工学研究科, Department of Computer Science, Tokyo Institute of Technology. コンピュータソフトウェア, Vol.23, No.3 (2006), pp.66–84.

[論文] 2004 年 7 月 12 日受付.

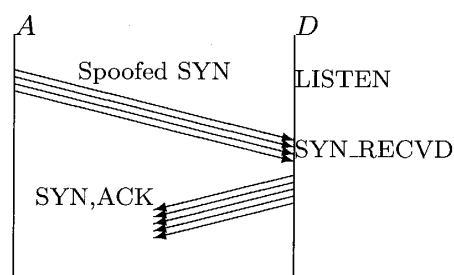
でデータを送信したのかを表わす数であり、初期シーケンス番号はその初期値であり、一般にランダムに決められる。

- 次に、接続先 D が接続を許可する場合は、SYN-ACK と呼ばれるパケットを S に返す。このパケットには、 D から S への一連の通信に対する初期シーケンス番号 y の他、 S から D への通信に対する応答確認番号 (acknowledgement number, ACK) が含まれる。この応答確認番号は、 S から D へ流れるデータに対する初期シーケンス番号 x に 1 を加えた数となっている。 D は、接続元のアドレス・ポート番号やシーケンス番号など、確立しようとする接続に関する情報を記憶しておかなければならない。
- SYN-ACK パケットを接続元 S が受けとると、 S は D に ACK パケットを送信する。この中には、シーケンス番号として $x+1$ 、応答確認番号として $y+1$ が含まれている。 D は、記憶しておいたこの接続に関する情報と ACK パケットの情報とを比較しチェックする。 D は SYN-ACK パケットを送出後、 S からの ACK パケットを一定時間待つ。



もし、攻撃者 A がアドレスを偽装し送信元アドレスを変えつつ、SYN パケットを送ったとする。そうすると、 D は SYN-ACK パケットを擬装された送信元に対して送信し、一定時間その ACK パケットの返信を待つ。しかしながら、その擬装された送信元からの返信はない。もし短時間にこのような擬装された SYN パケットが多量に送信されたとなると、 D における接続情報を格納する記憶領域があふれ、TCP のサービスが停止することとなる。これが、SYN あふ

れ攻撃である。



SYN cookie など、SYN あふれ攻撃にはさまざまな対策 [12] が提唱されている。

1.2 サービス不能攻撃の形式化

サービス不能攻撃は、さまざまな側面から研究が進んでおり、例えば、資源管理によるサービス不能攻撃からの防御 [10] や、プロトコルにおけるサービス不能攻撃耐性の向上 [3] [4] などの研究がある。また、メドーズは、サービス不能攻撃耐性を解析するための形式的体系 [8] [9] を提唱した。この形式的体系は、アリス・ボブ記法 (Alice-and-Bob notation) に計算コストに関する情報を付記するという拡張を行なっている。論文 [8] では、ディフィー・ヘルマンのプロトコルの一種である STS プロトコル (Station-to-Station protocol) を例にあげている。

- $A \rightarrow B: \alpha^{X_A}$
- $B \rightarrow A: \alpha^{X_B}, E_K(S_B(\alpha^{X_B}, \alpha^{X_A}))$
- $A \rightarrow B: E_K(S_A(\alpha^{X_A}, \alpha^{X_B}))$

但し、 α は、 A と B が共有する素数 p の原始根であり、 X_A と X_B は、 A と B のそれぞれがもつ秘密情報である。 K はディフィー・ヘルマンの共有鍵 $\alpha^{X_A X_B}$ である。 E_K は共有鍵 K による暗号化関数を表し、 S_A と S_B は、それぞれ A による署名、 B による署名を表す。メドーズはこれに対して、各ステップにおける処理に関する記述を付ける。

- $A \rightarrow B: \text{preexp}_1, \text{storename}_1 \parallel \alpha^{X_A} \parallel \text{storenonce}_1, \text{storename}_2, \text{accept}_1$
- $B \rightarrow A: \text{preexp}_2, \text{sign}_1, \text{exp}_1, \text{encrypt}_1 \parallel \alpha^{X_B}, E_K(S_B(\alpha^{X_B}, \alpha^{X_A})), \text{exp}_2, \text{checkname}_1, \text{retrievenonce}_1, \text{decrypt}_1, \text{checksig}_1, \text{accept}_2$

3. $A \rightarrow B$: $sign_2, encrypt_2 \parallel$
 $E_K(S_A(\alpha^{X_A}, \alpha^{X_B}))$
 $checkname_2, retrievenonce_2,$
 $decrypt_2, checksig_2, accept_4.$

但し、処理 $preexp$ は、静的に底がわかっている場合のべき乗であり、 exp は底が動的にしかわからない場合のべき乗である。処理 $storename$ と $storenonce$ は各々、名前の保存、ナンスの保存を意味する。

プロトコルの進行においてこれらの処理がどのようになされるかを追跡することにより、各プロセスの処理コストを計算する。例えば、攻撃者 I が B に対して、ランダムにでたらめな内容をもつメッセージを送信した場合には、被攻撃者 B は以下のような処理を実行しなければならない。

$storenonce_1, storename_2, accept_1, preexp_2,$
 $sign_1, exp_1, encrypt_1.$

この体系では、各処理に対してコストが割り当てられており、この場合、処理 $sign$ や処理 exp には、コスト $expensive$ が割り当てられ、処理 $preexp$ や処理 $encrypt$ には、コスト $intermediate$ が割り当てられ、 $storenonce$ や $storename$ にはコスト $cheap$ が割当てられている。実行される処理の列から、コストの総計を計算すると、

$$2expensive + 2medium + 2cheap,$$

となる。これにより、被攻撃者 B が要するコストが、攻撃者の要するコスト (ランダムなメッセージを生成・送信) より明らかに高価であることがわかる。

1.3 動機

メドーズの体系は、サービス不能攻撃体制の解析のための最初の形式的枠組みであり、アリス・ボブ記法に基づいていて理解しやすいという利点をもつ。しかしながら、メドーズのコスト解析は、プロトコル記述に対する処理に関する付記に基づいており、どのように処理を付記するのかはプロトコル記述を行なう者の責任となる。もし、処理の付記を間違えば、コスト解析も誤った結果となる。しかし、プロトコルの進行において行なわれている処理は、本質的には、プロトコルのメッセージ仕様自体に非明示的ではあるものの、記述されている。そこで、われわれは、プロ

トコルの操作的意味に基づくコスト計算を行なうことが可能となるようなプロトコル記述体系を確立することを目指す。

安全プロトコル記述体系として、例えば、spi 計算 (secure pi-calculus, spi-calculus) [1][2][16] がアバディとゴードンにより、提唱されている。ゴードンとジェフリーは、この spi 計算を用いた型理論的な手法 [5][6][7] で、認証性の検証手法を提唱している。

論文 [13] では、spi 計算に対してコスト計測の機構を拡張し、サービス拒否攻撃を記述・解析することを可能とする形式的体系 spice 計算 (secure pi-calculus for cost estimation) を提唱した。この体系は、spi 計算を拡張したもので、プロセスの計算における計算コストが、サーバーやクライアントの計算機において、どのように費されるかを明示的に表現できるように、システムにおける計算機構成を型として形式化した。書き換えスタイルで操作的意味論をあたえ、プロセスに対する型付けの情報を利用することにより、計算の進行における計算コストを区別するようになっていく。記憶コストは、サービス不能攻撃耐性を測る場合、各種の計算コストのうちで最も重要となるのだが、spice 計算では、記憶領域の解放を明示的に行なうようにした。本論文では、従来の spice 計算における型体系と操作的意味論を、計算機ごとの記憶コストの見積りに併せて、記憶領域の解放に関する正当性を保証するように、改良した。

以下では、spice 計算の構文と操作的意味論を紹介した後、型体系を与える。そして、記憶コスト健全性、すなわち、記憶コストが通算すると負にならないという性質を示す。最後に、SYN あふれ攻撃とその防御策である SYN クッキーが形式化できることを適用例として紹介する。

2 コスト解析のための体系

名前の可算集合 $\mathcal{N}$ と変数の可算集合 $\mathcal{V}$ が与えられているとする。名前と変数から、まず、値と呼ばれる構文を定義し、それを拡大した構文として、項を定義する。

定義 1 (項と値) 値は以下のように帰納的に定義される。

$V, U, W ::=$	値
n	名前
x	変数
i	整数
$(V_1, \dots, V_n)$	対値
$hash_V$	ハッシュ値

そして、項は以下のように帰納的に定義される。

$M, N ::=$	項
V	値
$(M + N)$	加法
$[M_1, \dots, M_n]$	対式
$hash(M)$	ハッシュ式

ハッシュに対して、値と式が両方定義されているが、式は評価前の構文を表し、値は評価後の結果であるデータを表す。次にプロセスと呼ばれる構文を定義する。名前はプロセス間通信の際のポート名を表す。変数は、データを格納する領域を表す。対式は、メッセージ列を表現するのに用いられる。

定義 2 (プロセス) プロセス (process) は以下のよう

$P, Q, R ::=$	プロセス
$\text{out } M_{port} \langle N_{msg} \rangle; P$	出力
$\text{inp } M_{port} (x); P$	入力
$(P \mid Q)$	合成
$\text{new}(n); P$	名前制限
$\text{repeat } P$	複製
stop	空
$\text{store } x = M; P$	記憶割り当て
$\text{free } x; P$	記憶解放
$\text{match } M \text{ is } N \text{ err}\{P\}; Q$	マッチング
$\text{split } [x_1, \dots, x_n] \text{ is } M \text{ err}\{R\}; P$	

出力 (プロセス) は、第一引数 M_{port} が通信ポート名、第二引数 N_{msg} が送信メッセージ、第三引数 P が継続されるプロセスを表す。入力 (プロセス) は、第一引数 M_{port} が通信ポート名、第二引数で与えられる変数 x が受信メッセージを格納する変数、第三引数 P が継続されるプロセスを表す。合成は並列実行を

表す。名前制限は新しい名前の生成を表す。複製は、プロセスの複製の生成を表す。記憶割り当ては記憶領域の新規割り当てを表し、記憶解放は、割り当てられた記憶の解放を表す。記憶領域の新規割り当ては、記憶割り当ての他、入力や対分解で行なわれる。

論文 [13] でおこなったように、暗号化、復号化、署名なども同様に定義できるが、後で利用しないので本論文では議論しない。

変数を束縛するものとして、入力、対分解、記憶割り当てがある。束縛されない変数を自由変数と呼ぶ。一方、名前を束縛するのは名前制限である。束縛されない名前を自由名前と呼ぶ。形式的には、以下のよう

定義 3 (自由変数) プロセス P における自由変数の集合 $fv(P)$ は、以下のように帰納的に定義される。

$$\begin{aligned}
 fv(\text{out } M \langle N \rangle; P) &= fv(M) \cup fv(N) \cup fv(P), \\
 fv(\text{inp } M (x); P) &= fv(M) \cup fv(P) \setminus \{x\}, \\
 fv(P \mid Q) &= fv(P) \cup fv(Q), \\
 fv(\text{new}(n); P) &= fv(P), \\
 fv(\text{repeat } P) &= fv(P), \\
 fv(\text{stop}) &= \emptyset, \\
 fv(\text{store } x = M; P) &= fv(P) \setminus \{x\}, \\
 fv(\text{free } x; P) &= fv(P),
 \end{aligned}$$

$$\begin{aligned}
 fv(\text{match } M \text{ is } N \text{ err}\{P\}; Q) \\
 &= fv(M) \cup fv(N) \cup fv(P) \cup fv(Q), \\
 fv(\text{split } [x_1, \dots, x_n] \text{ is } M \text{ err}\{R\}; P) \\
 &= fv(M) \cup (fv(P) \setminus \{x_1, \dots, x_n\}) \cup fv(R),
 \end{aligned}$$

ただし、項 M 中の自由変数の集合 $fv(M)$ は、項 M に出現する変数すべての集合として定義する。spice 計算では、変数を束縛するのは、入力、対分解、記憶割り当てである。

定義 4 (自由名前) プロセス P の自由名前の集合 $fn(P)$ は以下のように帰納的に定義される。制限プロセスに対しては、

$$fn(\text{new}(n); P) = fn(P) \setminus \{n\}$$

と定義される。その他のプロセスについては、似かよったものとなる。例えば、出力、合成、対分解に対しては以下のように定義される。

$$\begin{aligned}
& fn(out\ M\ \langle N \rangle; P) \\
&= fn(M) \cup fn(N) \cup fn(P), \\
& fn(P \mid Q) = fn(P) \cup fn(Q), \\
& fn(split\ [x_1, \dots, x_n]\ is\ M\ err\{R\}; P) \\
&= fn(M) \cup fn(P) \cup fn(R),
\end{aligned}$$

ただし、項 M に対する自由名前の集合 $fn(M)$ は、項 M に出現する名前すべての集合として定義される。spice 計算で、名前に対して束縛するのは制限 $new(n)$ のみである。

pi 計算の操作的意味論 [11] には、反応規則 (reaction rule) に基づくものと、ラベル付き遷移関係を拡張したコミットメント規則 (commitment rule) に基づくものがある。本論文で与える Spice 計算はコミットメント規則に基づく操作的意味論を拡張する。そのために、エージェントと呼ばれる構文を定義する。これらを直感的に説明すると、出力プロセスや入力プロセスが、送信シグナルや受信シグナルを送出した後、メッセージが変数に代入されるまでの中間状態を表す。

定義 5 (エージェント) エージェント (agent) は以下のように定義される。

$$\begin{array}{ll}
A, B ::= & \text{エージェント} \\
P & \text{プロセス} \\
(x)P & \text{抽象化} \\
(\nu n_1 \dots \nu n_j) \langle M \rangle P & \text{具体化}
\end{array}$$

抽象化 $(x)P$ を表すメタ変数として、 $F, F', \dots$ を用い、具体化 $(\nu n_1 \dots \nu n_j) \langle M \rangle P$ を表すメタ変数として、 $C, C', \dots$ を用いる。

プロセスと同様に、エージェントに対して、名前制限 $new(n); A$ と合成 $A \mid B$ を定義する。これらは共に、いわゆる「マクロ」である。

定義 6 (エージェント上の名前制限) エージェント上の名前制限は以下の等式により帰納的に定義される。

$$\begin{aligned}
& new(n); (x)P \equiv (x)(new(n); P), \\
& n \notin \{n_1, \dots, n_j\} \text{ をみたす名前 } n \text{ に対して,} \\
& new(n); ((\nu n_1 \dots \nu n_j) \langle M \rangle P) \\
& \equiv (\nu n_1 \dots \nu n_j) \langle M \rangle P \quad (n \notin fn(M) \text{ のとき}) \\
& \equiv (\nu n_1 \dots \nu n_j) \langle M \rangle new(m); P \quad (\text{その他})
\end{aligned}$$

定義 7 (エージェント上の合成) エージェント上の合成は以下の等式により帰納的に定義される。

$$\begin{aligned}
& x \notin fv(R) \text{ をみたすとき} \\
& R \mid ((x)P) \equiv (x)(R \mid P) \\
& ((x)P) \mid R \equiv (x)(P \mid R) \\
& \{n_1, \dots, n_j\} \cap fn(P) = \emptyset \text{ をみたすとき} \\
& ((\nu n_1 \dots \nu n_j) \langle M \rangle R) \mid P \\
& \equiv (\nu n_1 \dots \nu n_j) \langle M \rangle (R \mid P) \\
& P \mid ((\nu n_1 \dots \nu n_j) \langle M \rangle R) \\
& \equiv (\nu n_1 \dots \nu n_j) \langle M \rangle (P \mid R)
\end{aligned}$$

次に型を定義する。変数の集合の他、計算機名の集合があらかじめ与えられていると仮定する。計算機名を表すメタ変数として、 a, b, c などを使用する。

定義 8 (型) 型は次のように帰納的に定義される。

$$\begin{array}{ll}
A, B ::= & \text{型} \\
a :: \{x_1, \dots, x_n\} & \text{計算機型} \\
(A \mid B) & \text{合成型}
\end{array}$$

以下では、 $\mathcal{V}, \mathcal{V}'$ などは変数の有限集合を表すこととする。型 A に出現する変数全体の集合を $var(A)$ と表し、以下のように帰納的に定義される。

$$\begin{aligned}
var(a :: \mathcal{V}) &= \mathcal{V} \\
var(A \mid B) &= var(A) \cup var(B)
\end{aligned}$$

計算機型 $a :: \{x_1, \dots, x_n\}$ は、現在変数 $x_1, \dots, x_n$ が割当てられている計算機 a を表し、合成 $(A \mid B)$ は、計算機 A と B の並列合成、すなわち、 A と B との仮想的な並列計算機を表している。

Spice 計算の型付けは、大まかに分けて以下のような二つの役割がある。

計算コストの発生源追跡のための目印 プロセスの実行過程において、計算コストがどの計算機で発生したかを追跡するために、プロセスのどの部分がどの計算機で実行されるのかを、型付けにより記述する。

各計算機上で割り当てられる変数の制御 Spice 計算においては変数の割り当てを制御することにより、割り当てられていない記憶に対する記憶解放を制限する。

型付けは、以下で定義される型付け判定式 (typing judgement) $A \triangleright P$ により与えられる。これはプロセス P が型 A が表す計算機上で実行されるということの意味する。たとえば、 $(a :: \{x, y\} \mid b :: \{z\}) \triangleright (P \mid Q)$ は、計算機 a に変数 x, y が割り当てられていて、計算

機 $\mathbf{b}$ に変数 z が割り当てられていて、プロセス P はそのような状態の計算機 $\mathbf{a}$ 上で実行可能であり、 Q はそのような状態の $\mathbf{b}$ 上で実行可能であることを表す。
定義 9 (型付け判定式, 型付け規則) 型付け判定式は、型 A とエージェント A との間の二項関係であり、 $A \triangleright P$ と書き、以下の型付け規則により帰納的に定義される。

$$\text{TypeOut} \quad \frac{\mathbf{a}::\mathcal{V} \triangleright P \quad \mathcal{V} \supseteq \text{fv}(M) \quad \mathcal{V} \supseteq \text{fv}(N)}{\mathbf{a}::\mathcal{V} \triangleright \text{out } M \langle N \rangle; P}$$

$$\text{TypeIn} \quad \frac{\mathbf{a}::(\{x\} \cup \mathcal{V}) \triangleright P \quad \mathcal{V} \not\ni x \quad \mathcal{V} \supseteq \text{fv}(M)}{\mathbf{a}::\mathcal{V} \triangleright \text{inp } M(x); P}$$

入力プロセスの $\text{inp } M(x); P$ の型とその部分プロセス P の型とを比較すると、変数 x が追加されている点が異なるが、これは、プロセス P の実行が開始される時点で、変数 x が割り当てられていることを意味している。

$$\text{TypeComp} \quad \frac{A \triangleright P \quad B \triangleright Q}{(A | B) \triangleright (P | Q)}$$

$$\text{TypeRestr} \quad \frac{A \triangleright P}{A \triangleright \text{new}(n); P}$$

$$\text{TypeRepl} \quad \frac{\mathbf{a}::\emptyset \triangleright P}{\mathbf{a}::\emptyset \triangleright \text{repeat } P}$$

この規則は、プロセスの複製は変数が割り当てられていない状態の計算機上でのみ許されることを意味している。この制限により、プロセスの複製が実行されるときには、変数の格納領域は空になっていて、このとき、変数領域の複製は引き起されない。このように定義した理由は、プロセス複製のコスト付けを単純化するためである。

$$\text{TypeNil} \quad \frac{}{\mathbf{a}::\emptyset \triangleright \text{stop}}$$

この規則は、プロセス正常終了時には、変数が割り当てられていない状態でなければならないことを意味している。すなわち、この規則は、正常終了するときには、割り当てられた記憶領域はすべて、 free で開放されることを強制している。

$$\text{TypeStore} \quad \frac{\mathbf{a}::(\{x\} \cup \mathcal{V}) \triangleright P \quad \mathcal{V} \not\ni x \quad \mathcal{V} \supseteq \text{fv}(M)}{\mathbf{a}::\mathcal{V} \triangleright \text{store } x = M; P}$$

$$\text{TypeFree} \quad \frac{\mathbf{a}::\mathcal{V} \triangleright P \quad \mathcal{V} \not\ni x}{\mathbf{a}::(\{x\} \cup \mathcal{V}) \triangleright \text{free } x; P}$$

$$\text{TypeMatch} \quad \frac{\mathbf{a}::\mathcal{V} \triangleright P \quad \mathbf{a}::\mathcal{V} \triangleright Q \quad \mathcal{V} \supseteq \text{fv}(M) \quad \mathcal{V} \supseteq \text{fv}(N)}{\mathbf{a}::\mathcal{V} \triangleright \text{match } M \text{ is } N \text{ err}\{P\}; Q}$$

$$\text{TypeSplit} \quad \frac{\left\{ \begin{array}{l} \mathbf{a}::(\{x_1, \dots, x_n\} \cup \mathcal{V}) \triangleright P \\ \mathbf{a}::\mathcal{V} \triangleright R \\ \{x_1, \dots, x_n\} \cap \mathcal{V} = \emptyset \\ \mathcal{V} \supseteq \text{fv}(M) \end{array} \right.}{\mathbf{a}::\mathcal{V} \triangleright \text{split } (x_1, \dots, x_n) \text{ is } M \text{ err}\{R\}; P}$$

$$\text{TypeAbstr} \quad \frac{A \triangleright P}{A \triangleright (x)P}$$

$$\text{TypeConcr} \quad \frac{A \triangleright P \quad \text{fv}(M) = \emptyset}{A \triangleright (\nu n_1 \dots n_j) \langle M \rangle P}$$

式 M には自由変数が含まれていないことが条件になっている。後述の操作的意味論からわかるように、この具体化エージェントが発生するときには、実際には、式 M は、自由変数を含まないような値となる。

例えば、プロセス

$$\text{inp } n(x); \text{out } n \langle x+1 \rangle; \text{stop}$$

は通信ポート n から整数を受信し、それに 1 をたしたものを、そのポートから送信して、停止するという処理を行う。変数 x を解放していないので、このプロセスには型が付かない。実際、

$$\mathbf{a}::\mathcal{V} \triangleright \text{inp } n(x); \text{out } n \langle x+1 \rangle; \text{stop}$$

であると仮定する。型付け規則 **TypeIn** により、

$$\mathbf{a}::\{x\} \cup \mathcal{V} \triangleright \text{out } n \langle x+1 \rangle; \text{stop}$$

でなければならず、型付け規則 **TypeOut** により、

$$\mathbf{a}::\{x\} \cup \mathcal{V} \triangleright \text{stop}$$

が成り立たなければならないことになるが、これは型付け規則 **TypeNil** に矛盾する。一方、停止直前に変数 x を解放すれば型は付く。

$$\mathbf{a}::\emptyset \triangleright \text{inp } n(x); \text{out } n \langle x+1 \rangle; \text{free } x; \text{stop}$$

ここで与える型付けの導出木はプロセスに対して一意的に定まる。

命題 1 プロセス P に対して、型付け可能ならば、型付けを導く導出木は一意的である。

この命題は型付けの導出木の構造に関する帰納法に

より容易に証明される。

エージェントに対する名前制限 $\text{new}(n); A$, 合成 $(A \mid B)$ に対しても, 型付け規則 **TypeRestr**, **TypeComp** と同等の性質が成り立つ。

命題 2 1. $A \triangleright A$ ならば $A \triangleright \text{new}(n); A$.
2. $A \triangleright A$ かつ $B \triangleright B$ ならば, $(A \mid B) \triangleright (A \mid B)$.

変数情報は無視して, 同一の計算機同士の合成は一つの計算機と同一視するというのが, 以下の型同値である。

定義 10 (型同値) 型の間の二項関係 $A \simeq B$ を以下の規則から帰納的に定義する。

$$\frac{}{a::V \simeq a::W} \quad \frac{}{(a::V) \mid a::W \simeq a::X}$$

$$\frac{}{A \simeq A} \quad \frac{A \simeq A'}{(A \mid B) \simeq (A' \mid B)} \quad \frac{B \simeq B'}{(A \mid B) \simeq (A \mid B')}$$

2.1 コスト

コストは, 数値や Meadows[8][9] のように順序集合の元として定義することも考えられるが, 本論文では, 各種の処理を何回行なったかということを表す式を導入する。

コスト基底は, コストの種類を表す最小単位である。

定義 11 (コスト基底) コスト基底 (cost base) として, 本論文では以下のものを用いる。 $\mathbf{u}, \mathbf{v}, \mathbf{w}$ をメタ変数として用いる。

store_x (変数 x への記憶), pair (対生成),
 hash (ハッシュ値の計算), match (マッチング), repeat (プロセス複製)。

定義 12 (コスト値) コスト値 (cost value) は, コスト基底を整数係数で線型結合したものである。すなわち, $\mathbf{u}_1, \dots, \mathbf{u}_j$ をコスト基底とし, $n_1, \dots, n_j$ を整数とするならば,

$$n_1 \mathbf{u}_1 + \dots + n_j \mathbf{u}_j$$

はコスト値となる。コスト値の集合は, コスト基底により自由生成された加法群である。

定義 13 (コスト割り当て) コスト割り当ては, 型とコストの対の有限集合である。すなわち, $\mathbf{a}_1, \dots, \mathbf{a}_n$ を計算機名, $c_1, \dots, c_n$ をコスト値とすると, コスト割り当ては次のように書かれる。

$$\{\mathbf{a}_1 \cdot c_1, \dots, \mathbf{a}_n \cdot c_n\}.$$

コスト値の (ラベル付き) 直積とみなして, コスト割り当てにも加法が導入される。例えば,

$$\begin{aligned} & \{\mathbf{a} \cdot (2\text{pair} + \text{hash}), \mathbf{b} \cdot \text{match}\} \\ & + \{\mathbf{a} \cdot (\text{pair} + 3\text{hash}), \mathbf{b} \cdot \text{enc}\} \\ & = \{\mathbf{a} \cdot (3\text{pair} + 4\text{hash}), \mathbf{b} \cdot (\text{match} + \text{enc})\} \end{aligned}$$

次に項に対する評価を定義する。これは, 項に関する計算を形式化したものである。

定義 14 (項の評価) 項の評価 $M \downarrow V : c$ は, 項 M , 値 V , コスト値 c との間の三項関係であり, 以下の規則によって定義される。

$$\frac{}{V \downarrow V : c}$$

$$\frac{M_1 \downarrow i_1 : c_1 \quad M_2 \downarrow i_2 : c_2}{(M_1 + M_2) \downarrow (i_1 + i_2) : c_1 + c_2}$$

$$\frac{M_1 \downarrow V_1 : c_1 \quad \dots \quad M_n \downarrow V_n : c_n}{[M_1, \dots, M_n] \downarrow (V_1, \dots, V_n) : c_1 + \dots + c_n + \text{pair}}$$

$$\frac{M \downarrow V : c}{\text{hash}(M) \downarrow \text{hash}_V : c + \text{hash}}$$

次にプロセスの簡約関係を定義する。これは, 同一の計算機で実行される計算を定式化したものである。spi 計算と spice 計算とで最も異なるのは, 変数の機構である。spice 計算では, 変数の解放を明示的に書くことを要求する。それにより, 詳細な記憶コストの解析が可能となっている。変数の参照機構は, λ 計算や pi 計算では, 代入により形式化されている。例えば, プロセス $\text{free } x; P$ が変数 x に値 V が束縛されている状態で実行されるという状況を考えよう。変数 x の解放された後のプロセス P においては, この束縛 $x \mapsto V$ は有効ではない。したがって, 代入 $[V/x]$ も, P には適用されるべきではない。このことを踏まえると以下のように代入は形式的に定義される。

定義 15 (代入) プロセス P における変数 x に対する項 L の代入 $P[L/x]$ は以下のように帰納的に定義される。

$$\begin{aligned} & (\text{out } M \langle N \rangle; P)[L/x] \\ & = \text{out } M[L/x] \langle N[L/x] \rangle; P[L/x] \\ & (\text{inp } M(x); P)[L/x] = \text{inp } M[L/x](x); P \\ & (\text{inp } M(y); P)[L/x] = \text{inp } M[L/x](y); P[L/x] \end{aligned}$$

$$\begin{aligned}
(P \mid Q)[L/x] &= P[L/x] \mid Q[L/x] \\
(\text{new}(n); P)[L/x] &= \text{new}(n); P[L/x] \\
(\text{repeat } P)[L/x] &= \text{repeat } P[L/x] \\
\text{stop}[L/x] &= \text{stop} \\
(\text{match } M \text{ is } N \text{ err}\{P\}; Q)[L/x] \\
&= \text{match } M[L/x] \text{ is } N[L/x] \text{ err}\{P[L/x]\}; \\
&\quad Q[L/x](\text{split } [x_1, \dots, x_n] \text{ is } M \text{ err}\{R\}; \\
&\quad P)[L/x] \\
&= \text{split } [x_1, \dots, x_n] \text{ is } M[L/x] \text{ err}\{R[L/x]\}; \\
&\quad P[L/x](\text{decrypt } M \text{ is enc}(x', N) \text{ err}\{R\}; \\
&\quad P)[L/x] \\
&= \text{decrypt } M[L/x] \text{ is enc}(x', N[L/x]) \\
&\quad \text{err}\{R[L/x]\}; P[L/x] \\
&= (\text{decrypt } M \text{ is pubenc}(x', N) \text{ err}\{R\}; P)[L/x] \\
&= \text{decrypt } M[L/x] \text{ is pubenc}(x', N[L/x]) \\
&\quad \text{err}\{R[L/x]\}; P[L/x] \\
&= (\text{verify } M \text{ is sign}(x', N) \text{ err}\{R\}; P)[L/x] \\
&= \text{verify } M[L/x] \text{ is sign}(x', N[L/x]) \\
&\quad \text{err}\{R[L/x]\}; P[L/x] \\
&= (\text{store } y = M; P)[L/x] \\
&= (\text{store } y = M[L/x]; P[L/x]) \\
&= (\text{free } x; P)[L/x] = (\text{free } x; P) \\
&= (\text{free } x'; P)[L/x] = \text{free } x'; (P[L/x])
\end{aligned}$$

λ 計算や pi 計算のように、変数名の衝突を避けるように適切に束縛変数の名前替えが行なわれると仮定する。

定義 16 (プロセスの簡約関係) プロセス P, Q , コスト値 c の間の三項関係 $P > Q : c$ は以下の規則により帰納的に定義される。

RedRepeat

$$\frac{}{\text{repeat } P > P \mid (\text{repeat } P) : \text{repeat}}$$

RedStore

$$\frac{M \downarrow V : c \quad fv(V) = \emptyset}{\text{store } x = M; P > P[V/x] : c + \text{store}_x}$$

RedFree

$$\frac{}{\text{free } x; P > P : -\text{store}_x}$$

RedMatch

$$\frac{M \downarrow V : c \quad N \downarrow V : d}{\text{match } M \text{ is } N \text{ err}\{P\}; Q > Q : c + d + \text{match}}$$

RedMatchErr

$$\frac{M \downarrow V : c \quad N \downarrow W : d \quad V \neq W}{\text{match } M \text{ is } N \text{ err}\{P\}; Q > P : c + d + \text{match}}$$

RedSplit

$$\frac{M \downarrow (V_1, \dots, V_n) : c \quad fv(V_i) = \emptyset \ (i = 1, \dots, n)}{\text{split } [x_1, \dots, x_n] \text{ is } M \text{ err}\{R\}; P > P[V_1/x_1, \dots, V_n/x_n] : c + n \times \text{store}_x}$$

RedSplitErr

$$\frac{M \downarrow V : c \quad V \text{ is not a pair.}}{\text{split } (x_1, \dots, x_n) \text{ is } M \text{ err}\{R\}; P > R : c}$$

次に定義する複簡約関係は、上で簡約関係により定式化した単一計算機内計算が、複数の計算機で行なわれる処理を定式化し、さらに、その推移的閉包を取ったものである。

定義 17 (プロセス間の複簡約関係) 型 $\mathcal{A}$, プロセス P, Q , コスト割り当て σ の間の四項関係 $\mathcal{A} \vdash P \gg Q : \sigma$ は以下の規則により帰納的に定義される。

MRedRef

$$\frac{}{\mathcal{A} \vdash P \gg P : \{ \}}$$

MRedSingle

$$\frac{P > P' : c}{a : \mathcal{V} \vdash P \gg P' : \{a \cdot c\}}$$

MRedTrans

$$\frac{\mathcal{A} \vdash P \gg P' : \sigma_1 \quad \mathcal{A} \simeq \mathcal{B} \quad \mathcal{B} \vdash P' \gg P'' : \sigma_2}{\mathcal{A} \vdash P \gg P'' : \sigma_1 + \sigma_2}$$

MRedPar

$$\frac{\mathcal{A} \vdash P \gg P' : \sigma_1 \quad \mathcal{B} \vdash Q \gg Q' : \sigma_2}{(\mathcal{A} \mid \mathcal{B}) \vdash (P \mid Q) \gg (P' \mid Q') : \sigma_1 + \sigma_2}$$

MRedRestr

$$\frac{\mathcal{A} \vdash P \gg P' : \sigma}{\mathcal{A} \vdash \text{new}(n); P \gg \text{new}(n); P' : \sigma}$$

四項関係 $\mathcal{A} \vdash P \gg Q : \sigma$ は、直感的には、プロセス P を $\mathcal{A}$ で表される計算機構成の下で何ステップか実行するとプロセス Q に遷移し、その計算で要するコストが σ であることを意味する。

上記の複簡約関係は、複数の計算機での計算を扱ってはいるものの、あくまでも、同一計算機内での計算であり、プロセス間通信は行なわれない。以下で与えるコミットメント関係により、異なる計算機間でのプロセス間通信を定式化する。

定義 18 (プロセスのコミットメント関係) 型 $\mathcal{A}$, プロセス P , アクション α , エージェント A , コスト割り当て σ の間の五項関係 $\mathcal{A} \vdash P \xrightarrow{\alpha} A : \sigma$ は以下の規

則により帰納的に定義される。

CommIn

$$a::\mathcal{V} \vdash \text{inp } n(x); P \xrightarrow{n} (x)P : \{a \cdot \text{store}_x\}$$

CommOut

$$\frac{N \downarrow V : c \quad \text{fv}(V) = \emptyset}{a::\mathcal{V} \vdash \text{out } n \langle N \rangle; P \xrightarrow{\bar{n}} (\nu) \langle V \rangle; P : \{a \cdot c\}}$$

(注) 結論部の右辺の $(\nu) \langle V \rangle; P$ は, $n_1, \dots, n_j$ が一つもないような具体化のエージェント $(\nu n_1 \dots n_j) \langle V \rangle; P$ を意味する。

CommInter1

$$\frac{A \vdash P \xrightarrow{n} F : \sigma_1 \quad B \vdash Q \xrightarrow{\bar{n}} C : \sigma_2}{(A | B) \vdash (P | Q) \xrightarrow{\tau} F @ C : \sigma_1 + \sigma_2}$$

CommInter2

$$\frac{B \vdash Q \xrightarrow{\bar{n}} C : \sigma_2 \quad A \vdash P \xrightarrow{n} F : \sigma_1}{(B | A) \vdash (P | Q) \xrightarrow{\tau} C @ F : \sigma_2 + \sigma_1}$$

CommParLeft

$$\frac{A \vdash P \xrightarrow{\alpha} A : \sigma}{(A | B) \vdash (P | Q) \xrightarrow{\alpha} (A | Q) : \sigma}$$

CommParRight

$$\frac{B \vdash Q \xrightarrow{\alpha} A : \sigma}{(A | B) \vdash (P | Q) \xrightarrow{\alpha} (P | A) : \sigma}$$

CommRes

$$\frac{A \vdash P \xrightarrow{\alpha} A : \sigma \quad \alpha \notin \{m, \bar{m}\}}{A \vdash \text{new}(m); P \xrightarrow{\alpha} \text{new}(m); A : \sigma}$$

五項関係 $A \vdash P \xrightarrow{\alpha} A : \sigma$ は, 直感的には, A で表される計算機構成のもとでプロセス P を 1 ステップ実行すると, シグナル (送信・受信・送受信完了) α が発生し, プロセス Q に遷移し, その処理で要するコストが σ であることを意味する。

定義 19 (相互作用) エージェント F, C を各々 $(x)P$, $(\nu n_1, \dots, n_k) \langle M \rangle Q$ とする. 相互作用 $F @ C$ とは以下のように定義されるプロセスである. 相互作用 $C @ F$ についても, 左右対称に定義される。

$$\begin{aligned} (x)P @ (\nu n_1 \dots n_k) \langle M \rangle Q \\ &\equiv \text{new}(n_1); \dots \text{new}(n_k); (P[M/x] | Q) \\ ((\nu n_1 \dots n_k) \langle M \rangle Q) @ (x)P \\ &\equiv \text{new}(n_1); \dots \text{new}(n_k); (Q | P[M/x]) \end{aligned}$$

このように定義される相互作用は型に関して以下のような性質をもつ. これらの性質は, エージェントや相互作用の定義から明らか。

命題 3 (相互作用の型)

- $A \triangleright F$, かつ, $B \triangleright C$ ならば, $(A | B) \triangleright (F @ C)$.
- $(A | B) \triangleright (F @ C)$ ならば, $A \triangleright F$, かつ, $B \triangleright C$.
- $A \triangleright F$, かつ, $B \triangleright C$ ならば, $(B | A) \triangleright (C @ F)$.
- $(B | A) \triangleright (C @ F)$ ならば, $A \triangleright F$, かつ, $B \triangleright C$.

以下の命題は, コミットメント関係の導出木に関する帰納法で容易に証明される。

命題 4 $A \vdash P \xrightarrow{\tau} A : \sigma$ ならば, エージェント A はプロセスとなる。

以下で定義される複コミットメント関係は, 複簡約関係とコミットメント関係から定義されるもので, システム全体におけるプロトコルの実行を定式化したものである。

定義 20 (複コミットメント関係) 型 A , プロセス P, P' , コスト割り当て σ の間の四項関係 $A \vdash P \twoheadrightarrow P' : \sigma$ は以下の規則により帰納的に定義される。

MCommSingle

$$\frac{A \vdash P \xrightarrow{\tau} P' : \sigma}{A \vdash P \twoheadrightarrow P' : \sigma}$$

MCommAfterMRed

$$\frac{A \vdash P \twoheadrightarrow P' : \sigma_1 \quad A \simeq A' \quad A' \vdash P' \twoheadrightarrow P'' : \sigma_2}{A \vdash P \twoheadrightarrow P'' : \sigma_1 + \sigma_2}$$

MCommBeforeMRed

$$\frac{A \vdash P \twoheadrightarrow P' : \sigma_1 \quad A \simeq A' \quad A' \vdash P' \twoheadrightarrow P'' : \sigma_2}{A \vdash P \twoheadrightarrow P'' : \sigma_1 + \sigma_2}$$

MCommTrans

$$\frac{A \vdash P \twoheadrightarrow P' : \sigma_1 \quad A \simeq A' \quad A' \vdash P' \twoheadrightarrow P'' : \sigma_2}{A \vdash P \twoheadrightarrow P'' : \sigma_1 + \sigma_2}$$

2.2 主部簡約定理

次に, 主部簡約定理を証明する. これは, 簡約関係やコミットメント関係が成り立つプロセスの前後で, 型が保存されるという性質であり, Spice 計算においては, プロセスの計算機への配置が変化しないことを意味している. ただ, 計算機内においてはプロセスの複製がおこり, 一つの計算機で実行されるプロセスの個数は変化するし, また, 計算機で割当てられている変数も変化する. これらの差異を型同値により吸収する。

以下の補題は P の構造に関する帰納法により容易に証明される。

補題 1 もし $x \notin V$ かつ $a::V \triangleright P$ かつ $fv(V) = \emptyset$ ならば, $a::V \triangleright P[V/x]$.

注意: $a::V \triangleright P$ において, V はプロセス P で使用可能な変数を直接意味しているわけではなく, むしろ, 記憶解放 **free** により解放しうる変数を意味していることに注意されたい.

定理 1 (簡約関係に関する主部簡約定理) $P > Q : c$ かつ $A \triangleright P$ ならば, $A \simeq A'$ かつ $A' \triangleright Q$ をみたす A' が存在する.

証明 簡約関係 $P > Q : c$ の導出の構造に関する場合分けにより証明する. 導出木の最後に適用された規則により場合分けを行なう.

規則 **RedRepeat** の場合. $a::\emptyset \triangleright \text{repeat } P$ と $\text{repeat } P > (P \mid \text{repeat } P) : \text{repeat}$ を仮定する. 型同値の定義から, $a::\emptyset \simeq a::\emptyset \mid a::\emptyset$ である. 一方, 仮定と型付け規則 **TypeRepl** より, $a::\emptyset \triangleright P$. そして,

$$\frac{a::\emptyset \triangleright P \quad a::\emptyset \triangleright \text{repeat } P}{a::\emptyset \mid a::\emptyset \triangleright (P \mid \text{repeat } P)}$$

が成り立つ.

規則 **RedStore** の場合. $a::V \triangleright \text{store } x = M; P$ と $\text{store } x = M; P > P[V/x] : c + \text{store}_x$ を仮定する. 前者の仮定と型付け規則 **TypeStore** より, $a::(\{x\} \cup V) \triangleright P$, $V \not\ni x$ をえる. 一方, 後者の仮定と, 簡約関係の規則 **RedStore** より, $fv(V) = \emptyset$ をえる. 前補題より, $a::\{x\} \cup V \triangleright P[V/x]$.

規則 **RedSplit** は上の **RedStore** の場合と同様に証明される.

規則 **RedFree** の場合. $a::(\{x\} \cup V) \triangleright \text{free } x; P$ と $\text{free } x; P > P : -\text{store}_x$ を仮定する. 型付け規則 **TypeFree** より, $a::V \triangleright P$ かつ $x \notin V$ が成り立つ. 型同値の定義より, $a::\{x\} \cup V \simeq a::V$ であるので, この場合も命題は成り立つ.

規則 **RedMatch** の場合.

$$a::V \triangleright \text{match } M \text{ is } N \text{ err}\{P\}; Q$$

と

$$\text{match } M \text{ is } N \text{ err}\{P\}; Q > Q :: c + d + \text{match}$$

とを仮定する. 型付け規則 **TypeMatch** より, $a::V \triangleright Q$ である.

また, 規則 **RedMatchErr** の場合も上の

RedMatch と同様に証明される.

証終

定理 2 (複簡約関係に関する主部簡約定理) $A \triangleright P$, $A \simeq A'$, $A' \vdash P \gg Q : c$ ならば, $A \simeq A''$, $A'' \triangleright Q$ をみたす A'' が存在する.

証明 複簡約関係 $A \vdash P \gg Q : c$ の導出の構造に関する帰納法により証明する. 導出の最後の型付け規則により場合分けを行なう.

規則 **MRedRefl** の場合. 明らか.

規則 **MRedSingle** の場合. $a::V \vdash P \gg P' : \{a \cdot c\}$, $A' \triangleright P$, $a::V \simeq A'$ を仮定する. 前の命題により, $A' \simeq A''$, $A'' \triangleright P'$ をみたす A'' が存在する. 型同値の推移則により $A \simeq A''$.

規則 **MRedTrans** の場合.

$$\frac{\begin{array}{c} \vdots \\ A' \vdash P \gg P' : \sigma_1 \end{array} \quad \begin{array}{c} \vdots \\ A' \simeq B \quad B \vdash P' \gg P'' : \sigma_2 \end{array}}{A' \vdash P \gg P'' : \sigma_1 + \sigma_2}$$

と仮定する. 帰納法の仮定により, $A \simeq A''$ と $A'' \triangleright P'$ をみたす A'' が存在する. これにもう一つの帰納法の仮定を適用し, $A'' \simeq B'$ と $B' \triangleright P''$ をみたす B' が存在する. この B' は,

$$A \simeq A'' \simeq B'$$

をみたす.

規則 **MRedPar** の場合. $(A \mid B) \triangleright (P \mid Q)$, $(A \mid B) \simeq (A' \mid B')$,

$$\frac{\begin{array}{c} \vdots \\ A' \vdash P \gg P' : \sigma_1 \end{array} \quad \begin{array}{c} \vdots \\ B' \vdash Q \gg Q' : \sigma_2 \end{array}}{(A' \mid B') \vdash (P \mid Q) \gg (P' \mid Q') : \sigma_1 + \sigma_2}$$

とを仮定する. 帰納法の仮定より, $A \simeq A''$ かつ $A'' \triangleright P'$ をみたす A'' が存在し, $B \simeq B''$ かつ $B'' \triangleright Q'$ をみたす B'' が存在する. 型同値の推移則などにより, $(A \mid B) \simeq (A'' \mid B'')$ が成り立ち, 型付け規則 **TypeComp** により, $(A'' \mid B'') \triangleright (P' \mid Q')$ が成り立つ.

規則 **MRedRestr** の場合. $A \triangleright \text{new}(n); P$, $A \simeq A'$,

$$\frac{\begin{array}{c} \vdots \\ A' \vdash P \gg P' : \sigma \end{array}}{A' \vdash \text{new}(n); P \gg \text{new}(n); P' : \sigma}$$

を仮定する. 帰納法の仮定により, $A' \simeq A''$ と $A'' \triangleright P'$ をみたす A'' が存在する. 型付け規則 **TypeRestr** により, $A'' \triangleright \text{new}(n); P'$ が成り立つ.

証終

定理 3 (コミットメント関係に関する主部簡約定理)
 $A \triangleright P$, $A \simeq A'$, $A' \vdash P \xrightarrow{\alpha} A : \sigma$ ならば, $A \simeq A''$
 かつ $A'' \triangleright A$ をみたす A'' が存在する.

証明 コミットメント関係 $A' \vdash P \xrightarrow{\alpha} A : \sigma$ の導出の構造に関する帰納法により証明する. 導出の最後の規則に関して場合分けを行なう.

規則 **CommOut** の場合.

$$\frac{\begin{array}{c} \vdots \\ N \downarrow V : c \quad fv(V) = \emptyset \end{array}}{a::\mathcal{V} \vdash \text{out } n \langle N \rangle; P \xrightarrow{\bar{n}} (\nu) \langle V \rangle; P : \{a \cdot c\}}$$

と $a::\mathcal{V} \triangleright \text{out } n \langle N \rangle; P$ を仮定する. 型付け規則 **TypeOut** により, $a::\mathcal{V} \triangleright P$ が成り立つ. 一つ目の仮定より, $fv(V) = \emptyset$ である. よって, 型付け規則 **TypeConcr** より, $a::\mathcal{V} \triangleright (\nu) \langle V \rangle; P$ が成り立つ.

規則 **CommIn** の場合. $a::\mathcal{V}' \vdash \text{inp } n(x); P \xrightarrow{n} (x)P : \{a \cdot \text{store}_x\}$, $a::\mathcal{V} \triangleright \text{inp } n(x); P$ を仮定する. 型付け規則 **TypeIn** より, $a::\{x\} \cup \mathcal{V} \triangleright P$ が成り立つ. 型付け規則 **TypeAbstr** より, $a::\{x\} \cup \mathcal{V} \triangleright (x)P$ が成り立ち, この型 $a::\{x\} \cup \mathcal{V}$ は $a::\{x\} \cup \mathcal{V} \simeq a::\mathcal{V}$ をみたす.

規則 **CommInter1** の場合.

$$\frac{\begin{array}{c} \vdots \\ A \vdash P \xrightarrow{n} F : \sigma_1 \end{array} \quad \begin{array}{c} \vdots \\ B \vdash Q \xrightarrow{\bar{n}} C : \sigma_2 \end{array}}{(A | B) \vdash (P | Q) \xrightarrow{\tau} F @ C : \sigma_1 + \sigma_2}$$

と $(A | B) \triangleright (P | Q)$ を仮定する. 型付け規則 **TypeComp** より, $A \triangleright P$, かつ, $B \triangleright Q$. 帰納法の仮定より, $A \simeq A''$ かつ $A'' \triangleright P$ をみたす A'' と, $B \simeq B''$ かつ $B'' \triangleright Q$ をみたす B'' が存在する. 命題 3 より, $A'' | B'' \triangleright F @ C$ が成り立ち, この $A'' | B''$ は,

$$(A | B) \simeq (A'' | B'')$$

をみたす.

規則 **CommInter2** の場合. 上の場合と同様に証明される.

規則 **CommParLeft** の場合.

$$\frac{\begin{array}{c} \vdots \\ A' \vdash P \xrightarrow{\alpha} A : \sigma \end{array}}{(A' | B') \vdash (P | Q) \xrightarrow{\alpha} (A | Q) : \sigma}$$

と, $(A | B) \triangleright (P | Q)$ を仮定する. 型付け規則

TypeComp より, $A \triangleright P$ かつ $B \triangleright Q$. 帰納法の仮定より, $A \simeq A''$ かつ $A'' \triangleright P$ をみたす A'' が存在する. したがって, $(A'' | B) \triangleright (A | Q)$ が成り立ち, この型 $A'' | B$ は, $(A | B) \simeq (A'' | B)$ をみたす.

規則 **CommParRight** の場合. 上の場合と同様に証明される.

規則 **CommRes** の場合.

$$\frac{\begin{array}{c} \vdots \\ A' \vdash P \xrightarrow{\alpha} A : \sigma \quad \alpha \notin \{m, \bar{m}\} \end{array}}{A' \vdash \text{new}(m); P \xrightarrow{\alpha} \text{new}(m); A : \sigma}$$

と, $A \triangleright \text{new}(m); P$ を仮定する.

型付け規則 **TypeRestr** より, $A \triangleright P$ である. そして, 帰納法の仮定より, $A \simeq A''$, かつ, $A'' \triangleright A$ をみたす A'' が存在する. これから型付け規則 **TypeRestr** をもちいて, $A'' \triangleright \text{new}(m); A$ が導かれる.

証終

上記の定理から, 複コミットメント関係の導出木に関する帰納法を用いて容易に, 以下が導かれる.

定理 4 (複コミットメント関係の主部簡約定理)

$A \triangleright P$, $A \simeq A'$, $A' \vdash P \rightarrow P' : \sigma$ ならば, $A \simeq A''$ かつ $A'' \triangleright P'$ をみたす A'' が存在する.

2.3 記憶コストの健全性

プロセス $\text{free } x; \text{free } x; \text{free } x; \text{stop}$ を実行すると, 素朴には, 変数 x が3回解放されるので -3store_x となる. しかし, 実際にはこのプロセスには型が付かないように Spice 計算では型付けが与えられている. このように, 割り当てられていない変数を解放しないことを, 「記憶コストの健全性」と呼ぶこととする. 本節では, この記憶コストの健全性を示す.

型は, 計算機の構成と各計算機における変数の割り当て状態を表すことは既に説明した. 型 A が表す計算機の構成において変数 x が割り当てられている個数を $|A|_x$ で表す. 形式的には以下のように定義される.

定義 21 (型における変数の割り当て数) 型 A における変数 x の割り当て数 $|A|_x$ は以下のように, 型の構造に関して帰納的に定義される.

$$\begin{aligned} |a::\mathcal{V}|_x &= 1 \quad (x \in \mathcal{V}) \\ &= 0 \quad (x \notin \mathcal{V}) \end{aligned}$$

$$|(\mathcal{A} \mid \mathcal{B})|_x = |\mathcal{A}|_x + |\mathcal{B}|_x$$

コスト値 c やコスト割り当て σ が与えられたとき、その中に含まれる変数 x に関する記憶コストの量をおのの $|c|_x, |\sigma|_x$ で表す。

定義 22 (変数に関する記憶コスト量) コスト値 c , コスト割り当て σ の変数に関する記憶コスト量 $|c|_x, |\sigma|_x$ は以下の等式により帰納的に定義される。

$$|store_x|_x = 1$$

$$|u|_x = 0 \quad (u \neq store_x)$$

$$|n_1 u_1 + \dots + n_j u_j|_x = n_1 |u_1|_x + \dots + n_j |u_j|_x$$

$$|\{a_1 \cdot c_1, \dots, a_n \cdot c_n\}|_x = |c_1|_x + \dots + |c_n|_x$$

簡約関係・コミットメント関係の前後を比較すると、型における変数割り当て数が増加すると、同じ数だけ記憶コスト量が増加する。逆に、変数割り当て数が減少すると、その数だけ記憶コスト量も減少する。

定理 5 (簡約関係における記憶コスト健全性)

$P > Q : c$ かつ $\mathcal{A} \triangleright P$ かつ $\mathcal{A}' \triangleright Q$ ならば、

$$|\mathcal{A}'|_x - |\mathcal{A}|_x = |c|_x$$

証明 簡約関係の導出木に関する帰納法により証明する。導出木の最後に適用された規則により場合分けを行なう。

規則 **RedRepeat** の場合。型付規則 **TypeRepeat** より、 $\mathcal{A}, \mathcal{A}'$ 、ともに、

$$\mathcal{A} = \mathcal{A}' = a::\emptyset$$

という形をしている。よって、

$$|\mathcal{A}'|_x - |\mathcal{A}|_x = 0$$

であり、

$$|repeat|_x = 0$$

であるから、この定理は成り立つ。

規則 **RedStore** の場合。 $P = store\ y = M; P'$ (ただし $x \neq y$) の場合は、

$$|\mathcal{A}'|_x = |\mathcal{A}|_x = 0$$

である。そして、規則 **RedStore** から、 c はある c' に対して、 $c = c' + store_y$ と表されることがわかるが、 c' は項の評価で発生するコストで記憶コスト $store$ は含まないので、 $|c|_x = 0$ となる。よって、定理は成り立つ。

$P = (store\ x = M; P')$ の場合を考える。型付規則 **TypeStore** と補題 1 により、ある $\mathcal{V}, a$ に対して、

$$\mathcal{A} = a::\mathcal{V},$$

$$\mathcal{A}' = (a::\{x\} \cup \mathcal{V}),$$

$$x \notin \mathcal{V}$$

が成り立つ。よって、

$$|\mathcal{A}'|_x - |\mathcal{A}|_x = 1 - 0 = 1$$

である。一方、規則 **RedStore** より、ある c' に対し $c = c' + store_x$ が成り立つ。コスト値 c' は、項の評価において発生するもので、記憶コスト $store$ を含まない。よって、

$$|c|_x = |c' + store_x|_x = 1$$

が成り立ち、定理が成り立つ。

規則 **RedFree** の場合。 $P = free\ y; P'$ (ただし $x \neq y$) の場合は、

$$|\mathcal{A}'|_x = |\mathcal{A}|_x = 0$$

であり、

$$|c|_x = 0$$

であり、明らかに、この定理は成り立つ。

$P = free\ x; P'$ の場合を考える。

ある $\mathcal{V}, a$ に対して、

$$\mathcal{A} = (a::\{x\} \cup \mathcal{V}),$$

$$\mathcal{A}' = a::\mathcal{V},$$

$$x \notin \mathcal{V}$$

が成り立つ。よって、

$$|\mathcal{A}'|_x - |\mathcal{A}|_x = 0 - 1 = -1$$

一方、規則 **RedFree** により、 $c = -store_x$ であるから、 $|c|_x = -1$ 。したがって、この場合も定理は成り立つ。

規則 **RedMatch**, **RedMatchErr**, **RedSplitErr** の場合。これらの場合は、いずれも $\mathcal{A} = \mathcal{A}'$ であり、 $|c|_x = 0$ が成り立つので、明らかに定理は成り立つ。

規則 **RedSplit** の場合。上記の規則 **RedStore** の場合と同様に証明できる。

証終

この命題は複簡約関係に拡張できる。

定理 6 (複簡約関係における記憶コスト健全性)

$\tilde{A} \vdash P \gg Q : \sigma, \tilde{A} \simeq \mathcal{A}, \mathcal{A} \triangleright P, \mathcal{A}' \triangleright Q$ ならば、

$$|\mathcal{A}'|_x - |\mathcal{A}|_x = |\sigma|_x$$

証明 複簡約関係の導出木に関する帰納法により証明する。導出木の最後の規則で場合分けを行なう。

規則 **MRedRef** の場合。明らか。

規則 **MRedSingle** の場合。 $|c|_x = |a \cdot c|_x$ であるの

で, 前の定理 5 から明らか.

規則 **MRedTrans** の場合. $\sigma = \sigma_1 + \sigma_2$,

$$\frac{\begin{array}{c} \vdots \\ \tilde{A} \vdash P \gg P' : \sigma_1 \end{array} \quad \begin{array}{c} \vdots \\ A \simeq B \quad B \vdash P' \gg Q : \sigma_2 \end{array}}{\tilde{A} \vdash P \gg Q : \sigma_1 + \sigma_2},$$

$\tilde{A} \simeq A, A \triangleright P, A' \triangleright Q$ と仮定する. 複簡約関係に関する主部簡約定理 (定理 2) より, ある B' に対し, $B' \triangleright P'$ (かつ $A \simeq B'$) が成り立つ. 帰納法の仮定より,

$$|B'|_x - |A|_x = |\sigma_1|_x$$

であり,

$$|A'|_x - |B'|_x = |\sigma_2|_x$$

である. この二つの等式の両辺をたすと,

$$|A'|_x - |A|_x = |\sigma_1|_x + |\sigma_2|_x$$

をえる.

規則 **MRedPar** の場合.

$$\frac{\begin{array}{c} \vdots \\ \tilde{A}_1 \vdash P_1 \gg Q_1 : \sigma_1 \end{array} \quad \begin{array}{c} \vdots \\ \tilde{A}_2 \vdash P_2 \gg Q_2 : \sigma_2 \end{array}}{(\tilde{A}_1 \mid \tilde{A}_2) \vdash (P_1 \mid P_2) \gg (Q_1 \mid Q_2) : \sigma_1 + \sigma_2},$$

 $(\tilde{A}_1 \mid \tilde{A}_2) \simeq (A_1 \mid A_2), (A_1 \mid A_2) \triangleright (P_1 \mid P_2),$
 $(A'_1 \mid A'_2) \triangleright (Q_1 \mid Q_2),$ と仮定する. 型付規則 **TypeComp** より $A_1 \triangleright P_1, A_2 \triangleright P_2, A'_1 \triangleright Q_1, A'_2 \triangleright Q_2$ をえる. そして, 帰納法の仮定より,

$$|A'_1|_x - |A_1|_x = |\sigma_1|_x,$$

と

$$|A'_2|_x - |A_2|_x = |\sigma_2|_x$$

をえる. 両辺同士をたすと,

$$(|A'_1|_x + |A'_2|_x) - (|A_1|_x + |A_2|_x) = |\sigma_1|_x + |\sigma_2|_x,$$

となり,

$$|(A'_1 \mid A'_2)|_x - |(A_1 \mid A_2)|_x = |\sigma_1|_x + |\sigma_2|_x,$$

をえる.

規則 **MRedRestr** の場合. 明らか.

証終

定理 7 (コミットメント関係におけるコスト健全性)

$\tilde{A} \vdash P \xrightarrow{\alpha} A : \sigma, \tilde{A} \simeq A, A \triangleright P, A' \triangleright A$ ならば,

$$|A'|_x - |A|_x = |\sigma|_x.$$

証明 コミットメント関係の導出木の帰納法により証明される.

規則 **CommIn** の場合. $\mathbf{a}::\mathcal{V}' \vdash \text{inp } n(x); P \xrightarrow{n} (x)P : \{\mathbf{a} \cdot \text{store}_x\}, \mathbf{a}::\mathcal{V} \triangleright \text{inp } n(x); P, A' \triangleright (x)P$ と仮定する. 型付規則 **TypeIn** より, $\mathbf{a}::(\{x\} \cup \mathcal{V}) \triangleright P$

が成り立つ. 型付の一意性と型付規則 **TypeAbstr** より, $A' = \mathbf{a}::(\{x\} \cup \mathcal{V})$ をえる. したがって,

$$|\mathbf{a}::(\{x\} \cup \mathcal{V})|_x - |\mathbf{a}::\mathcal{V}|_x = 1 = |\{\mathbf{a} \cdot \text{store}_x\}|_x$$

となり, この定理は成り立つ.

規則 **CommOut** の場合は明らか.

規則 **CommInter1** の場合. $\sigma = \sigma_1 + \sigma_2$,

$$\frac{\begin{array}{c} \vdots \\ \tilde{A}_1 \vdash P \xrightarrow{n} F : \sigma_1 \end{array} \quad \begin{array}{c} \vdots \\ \tilde{A}_2 \vdash Q \xrightarrow{\bar{n}} C : \sigma_2 \end{array}}{(\tilde{A}_1 \mid \tilde{A}_2) \vdash (P \mid Q) \xrightarrow{\tau} F @ C : \sigma_1 + \sigma_2},$$

$(\tilde{A}_1 \mid \tilde{A}_2) \simeq (A_1 \mid A_2), (A_1 \mid A_2) \triangleright (P \mid Q),$
 $A' = (A'_1 \mid A'_2), (A'_1 \mid A'_2) \triangleright (F @ C),$ と仮定する.
 命題 3 より, $A'_1 \triangleright F, A'_2 \triangleright C$. 帰納法の仮定より,

$$|A'_1|_x - |A_1|_x = |\sigma_1|_x,$$

$$|A'_2|_x - |A_2|_x = |\sigma_2|_x$$

したがって,

$$(|A'_1|_x + |A'_2|_x) - (|A_1|_x + |A_2|_x) = |\sigma_1|_x + |\sigma_2|_x,$$

すなわち,

$$(|A'_1 \mid A'_2|_x) - (|A_1 \mid A_2|_x) = |\sigma_1 + \sigma_2|_x.$$

規則 **CommInter2** の場合. 上記, **CommInter1** の場合と同様.

規則 **CommParLeft** の場合.

$$\frac{\begin{array}{c} \vdots \\ \tilde{A}_1 \vdash P \xrightarrow{\alpha} A : \sigma \end{array}}{(\tilde{A}_1 \mid \tilde{A}_2) \vdash (P \mid Q) \xrightarrow{\alpha} (A \mid Q) : \sigma},$$

$(\tilde{A}_1 \mid \tilde{A}_2) \simeq (A_1 \mid A_2), (A_1 \mid A_2) \triangleright (P \mid Q),$
 $(A'_1 \mid A_2) \triangleright (A \mid Q)$ と仮定する. 帰納法の仮定により,

$$|A'_1|_x - |A_1|_x = |\sigma|_x$$

が成り立つ. よって,

$$(|A'_1|_x + |A_2|_x) - (|A_1|_x + |A_2|_x) = |\sigma|_x,$$

すなわち,

$$(|A'_1 \mid A_2|_x) - (|A_1 \mid A_2|_x) = |\sigma|_x.$$

規則 **CommParRight** の場合, **CommParLeft** の場合と同様.

証終

定理 8 (複コミットメント関係の記憶コスト健全性)

$\tilde{A} \vdash P \xrightarrow{\alpha} P' : \sigma, \tilde{A} \simeq A, A \triangleright P, A' \triangleright P'$ ならば,

$$|A'|_x - |A|_x = |\sigma|_x$$

が成り立つ.

注 特に, $|A|_x = 0$ の場合を考えると, $|A'|_x = |\sigma|_x$.

という等式をえる。これは $|\sigma|_x$ が負にならないことを意味し、これらの記憶コスト健全性から、変数の割り当てがない状態からプロセスの実行が始まる場合、記憶コストが負の値にならないことを示している。

証明 複コミットメント関係の導出木に関する帰納法により証明する。

規則 **MCommSingle** の場合は、コミットメント関係におけるコスト健全性より明らか。その他の場合は、前述の複簡約関係のコスト健全性の証明と同様である。

証終

3 Spice 計算の例

この章では、Spice 計算の例を紹介する。型 $a::\mathcal{V}$ にみられるように、型では、各計算機において割り当てられている変数の情報を管理していた。これは、記憶コストの健全性を示すことが目的であり、本章で考えるようなプロトコルのコスト解析においては、利用されない。したがって、この章では読みやすくするために、変数の情報の部分 $\mathcal{V}$ をすべて省略して考える。

以降の部分では次のような略記を用いる。

$$\begin{aligned} & \text{match } M_1 \text{ is } N_1 \text{ and } M_2 \text{ is } N_2 \text{ and } \dots \\ & \quad \text{and } M_n \text{ is } N_n \text{ err}\{R\}; P \\ \equiv & \text{ match } M_1 \text{ is } N_1 \text{ err}\{R\}; \\ & \quad \text{match } M_2 \text{ is } N_2 \text{ err}\{R\}; \dots \\ & \quad \text{match } M_n \text{ is } N_n \text{ err}\{R\}; P \\ & \quad \text{free } x_1, x_2, \dots, x_n \\ \equiv & \text{ free } x_1; \text{ free } x_2; \dots \text{ free } x_n; \text{ stop} \end{aligned}$$

また、以下で現われる式 $\text{succ}(M)$ は、 $M+1$ の略記であり、式 $\text{pred}(M)$ は、 $M-1$ の略記である。

3.1 3 ウエイハンドシェイク

以下は、3 ウエイハンドシェイクをアリスボブ記法をもちいて定式化したものである。

$$\begin{aligned} A \rightarrow B & : A, B, S_A \\ B \rightarrow A & : B, A, S_B, S_A + 1 \\ A \rightarrow B & : A, B, S_A + 1, S_B + 1. \end{aligned}$$

A と B は IP アドレスとポート番号を表したものであり、 S_A と S_B はシーケンス番号を形式化したものである。

ホスト A とホスト B は、各々プロセス P_A, P_B として形式化される。

$$\begin{aligned} P_A & \stackrel{\text{def}}{=} \text{ new}(S_A); \\ & \text{ store } x_{sa} = S_A; \\ & \text{ out } c \langle (A, B, x_{sa}) \rangle; \\ & \text{ inp } c(p); \\ & \text{ split } [x'_b, x'_a, x'_{sb}, x'_{sa1}] \text{ is } p \text{ err}\{\text{free } x_{sa}, p\}; \\ & \text{ free } p; \\ & \text{ match } x'_a \text{ is } A \text{ and } x'_b \text{ is } B \text{ and } x'_{sa1} \text{ is } \text{succ}(x_{sa}) \\ & \quad \text{err}\{\text{free } x_{sa}, x'_b, x'_a, x'_{sb}, x'_{sa1}, p\} \\ & \text{ free } x'_b, x'_a, x'_{sa1}; \\ & \text{ out } c \langle (A, B, \text{succ}(x_{sa}), \text{succ}(x'_{sb})) \rangle; \\ & P'_A, \end{aligned}$$

$$\begin{aligned} P_B & \stackrel{\text{def}}{=} \text{ new}(S_B); \\ & \text{ inp } c(q_1); \\ & \text{ split } [y_a, y_b, y_{sa}] \text{ is } q_1 \text{ err}\{\text{free } q_1\}; \\ & \text{ free } q_1; \\ & \text{ match } y_b \text{ is } B \text{ err}\{\text{free } y_a, y_b, y_{sa}\}; \text{ free } y_b; \\ & \text{ store } y_{sb} = S_B; \text{ out } c \langle (B, y_a, y_{sb}, \text{succ}(y_{sa})) \rangle; \\ & \text{ inp } c(q_2); \\ & \text{ split } [y'_a, y'_b, y'_{sa1}, y'_{sb1}] \text{ is } q_2 \\ & \quad \text{err}\{\text{free } y_a, y_{sa}, y_{sb}, q_2\}; \\ & \text{ free } q_2; \\ & \text{ match } y'_b \text{ is } B \text{ and} \\ & \quad y'_a \text{ is } y_a \text{ and} \\ & \quad y'_{sa1} \text{ is } \text{succ}(y_{sa}) \text{ and} \\ & \quad y'_{sb1} \text{ is } \text{succ}(y_{sb}) \\ & \quad \text{err}\{\text{free } y_a, y_{sa}, y_{sb}, y'_b, y'_a, y'_{sa1}, y'_{sb1}\} \\ & \text{ free } y'_b, y'_a, y'_{sa1}, y'_{sb1}; \\ & P'_B \end{aligned}$$

通常の通信が行なわれる場合のシステムは、以下のプロセス NormalConfig により記述される。

$$\text{NormalConfig} \stackrel{\text{def}}{=} (\text{repeat } P_A \mid \text{repeat } P_B).$$

そして、プロセス P_A, P_B が、各々計算機 $\mathbf{a}, \mathbf{b}$ 上で実行されるとすると、このプロセス NormalConfig は、 $\text{NormalConfig} : (\mathbf{a} \mid \mathbf{b})$ と型付けされる。

このプロセス NormalConfig を基点とする、コミットメント関係、および簡約関係の列は以下のようになる。

NormalConfig

$$\begin{aligned}
 &\equiv (\text{repeat } P_A \mid \text{repeat } P_B) \\
 &\xrightarrow{\tau} \text{new}(S_A); \text{new}(S_B); \\
 &\quad (\text{inp } c(p); \dots) \mid \text{repeat } P_A \\
 &\quad \mid (\text{split } [y_a, y_b, y_c] \text{ is } q_1; \dots) \mid \text{repeat } P_B \\
 &\quad : \{\mathbf{a} \cdot \text{store}, \mathbf{b} \cdot \text{store}\} \\
 &\gg \text{new}(S_A); \text{new}(S_B); \\
 &\quad ((\text{inp } c(p); \dots) \mid \text{repeat } P_A \\
 &\quad \mid (\text{out } c(\langle y_b, y_a, y_{sb}, \text{succ}(y_{sa}) \rangle); \dots) \\
 &\quad \mid \text{repeat } P_B) \\
 &\quad : \{\mathbf{b} \cdot 3\text{store}\} - \{\mathbf{b} \cdot \text{store}\} + \{\mathbf{b} \cdot \text{match}\} \\
 &\quad \quad + \{\mathbf{b} \cdot -\text{store}\} + \{\mathbf{b} \cdot \text{store}\} \\
 &\quad \quad = \{\mathbf{b} \cdot 2\text{store}, \mathbf{b} \cdot \text{match}\} \\
 &\xrightarrow{\tau} \text{new}(S_A); \text{new}(S_B); \\
 &\quad (\text{split } [x'_b, x'_a, x'_{sb}, x'_{sa1}] \text{ is } p \dots) \mid \text{repeat } P_A \\
 &\quad \mid (\text{inp } c(q_2); \dots) \mid \text{repeat } P_B \\
 &\quad : \{\mathbf{a} \cdot \text{store}\} \\
 &\gg \text{new}(S_A); \text{new}(S_B); \\
 &\quad (\text{out } c(\langle x_a, x_b, \text{succ}(x_{sa}), \text{succ}(x_{sb}) \rangle); \dots) \\
 &\quad \mid \text{repeat } P_A \mid (\text{inp } c(q_2); \dots) \mid \text{repeat } P_B \\
 &\quad : \{\mathbf{a} \cdot 4\text{store}\} + \{\mathbf{a} \cdot -\text{store}\} \\
 &\quad \quad + \{\mathbf{a} \cdot 3\text{match}\} - \{\mathbf{a} \cdot 3\text{store}\} \\
 &\quad \quad = \{\mathbf{a} \cdot 3\text{match}\} \\
 &\xrightarrow{\tau} \text{new}(S_A); \text{new}(S_B); \\
 &\quad (P_A \mid \text{repeat } P_A \\
 &\quad \quad \mid (\text{split } [y'_a, y'_b, y'_{sa1}, y'_{sb1}] \text{ is } q_2 \dots) \\
 &\quad \quad \mid \text{repeat } P_B) \\
 &\quad : \{\mathbf{b} \cdot \text{store}\} \\
 &\gg \text{new}(S_A); \text{new}(S_B); \\
 &\quad P'_A \mid \text{repeat } P_A \mid P'_B \mid \text{repeat } P_B \\
 &\quad : \{\mathbf{b} \cdot 4\text{store}\} + \{\mathbf{b} \cdot -\text{store}\} \\
 &\quad \quad + \{\mathbf{b} \cdot 4\text{match}\} + \{\mathbf{b} \cdot -4\text{store}\} \\
 &\quad \quad = \{\mathbf{b} \cdot -\text{store}, \mathbf{b} \cdot 4\text{match}\}
 \end{aligned}$$

結局、通常の3ウェイハンドシェイクの場合、以下のようにコストが費やされることがわかる。

NormalConfig

$$\begin{aligned}
 &\rightarrow (\text{new}(S_A); P'_A) \mid \text{repeat } P_A \\
 &\quad \mid (\text{new}(S_B); P'_B) \mid \text{repeat } P_B \\
 &\quad : \{\mathbf{a} \cdot 2\text{store}, \mathbf{a} \cdot 3\text{match}, \mathbf{b} \cdot 3\text{store}, \\
 &\quad \quad \mathbf{b} \cdot 5\text{match}\}
 \end{aligned}$$

この結果から、 A と B とのコストの差は無視できるといえる。

ホスト A は正常にコネクションの確立を要求するものであった。

3.2 SYN あふれ攻撃

次にSYNあふれ攻撃の場合を考える。ホスト I が B を攻撃する。

$$\begin{aligned}
 I \rightarrow B &: I_i, B, S_{I_i}, \\
 B \rightarrow I &: B, I_i, S_B, S_{I_i} + 1, \\
 &\quad (i = 1, 2, \dots)
 \end{aligned}$$

攻撃者 I は以下のプロセス P_I として定式化される。

$$P_I \stackrel{\text{def}}{=} \text{new}(i); \text{new}(s); \text{out } c(\langle i, B, s \rangle); \text{stop}.$$

被害者 B から攻撃者 I に送られるメッセージは、IPアドレスが擬装されるため、誰にも受信されることなくネットワークにおいて失なわれる。このようなメッセージの消失を形式化するために、ネットワークを表現するプロセス P_N を設ける。

$$P_N \stackrel{\text{def}}{=} \text{inp } c(r); \text{stop}.$$

攻撃者を表すプロセス P_I に計算機の型 i を割り当て、ネットワークを表すプロセス P_I に型 n を割り当て、被攻撃者となるプロセス P_B に型 b を割り当てるとする。このときの状況を表現するプロセス AttackConfig は以下のとおり。

$$\text{AttackConfig} \stackrel{\text{def}}{=} (\text{repeat } P_I \mid \text{repeat } P_B \mid \text{repeat } P_N).$$

このプロセスは以下のようなコミットメント関係・簡約関係の列をもつ。

AttackConfig

$$\begin{aligned}
 &\equiv (\text{repeat } P_I \mid \text{repeat } P_B \\
 &\quad \mid \text{repeat } P_N) \\
 &\xrightarrow{\tau} \text{new}(S_B); \\
 &\quad (\text{stop} \mid \text{repeat } P_I \\
 &\quad \quad \mid (\text{split } [y_a, y_b, y_c] \text{ is } q_1; \dots) \\
 &\quad \quad \mid \text{repeat } P_B \\
 &\quad \quad \mid \text{inp } c(r); \text{stop} \mid \text{repeat } P_N) \\
 &\quad : \{\mathbf{b} \cdot \text{store}\}
 \end{aligned}$$

```

>> new( $S_B$ );
  (stop | repeat  $P_I$ 
    | out  $c \langle (B, A, S_B, \text{succ}(y_{sa})) \rangle; \dots$ 
    | repeat  $P_B$ 
    | inp  $c(r)$ ; stop | repeat  $P_N$ )
  :  $\{\mathbf{b} \cdot 3\text{store}\} + \{\mathbf{b} \cdot -\text{store}\}$ 
  +  $\{\mathbf{b} \cdot \text{match}\} + \{\mathbf{b} \cdot -\text{store}\} + \{\mathbf{b} \cdot +\text{store}\}$ 
  =  $\{\mathbf{b} \cdot 2\text{store}, \mathbf{b} \cdot \text{match}\}$ 
>> new( $S_B$ );
  (stop | repeat  $P_I$  | inp  $c(q_2); \dots$ 
    | repeat  $P_B$  | stop | repeat  $P_N$ )
  :  $\{\mathbf{n} \cdot \text{store}\}$ 

```

したがって、この場合のコストは以下ようになる。

```

AttackConfig  $\rightarrow \tau$  new( $S_B$ );
  (stop | repeat  $P_I$  | inp  $c(q_2); \dots$ 
    | repeat  $P_B$  | stop | repeat  $P_N$ )
  :  $\{\mathbf{b} \cdot 4\text{store}, \mathbf{b} \cdot \text{match},$ 
     $\mathbf{n} \cdot \text{store}\}$ 

```

この結果から、攻撃者のコストは無視できるぐらい少ないのに対して、被害者のコストは、攻撃者からの送信回数に比例し、高価であると言える。

3.3 SYN クッキーによる SYN あふれ攻撃の防御

次に、SYN あふれ攻撃からの防御法のうちで、SYN クッキーという技法を Spice 計算で形式化する。

```

 $A \rightarrow B$  :  $A, B, S_A$ 
 $B \rightarrow A$  :  $B, A, H(A, B, S_A, \text{secret}, \text{count}), S_A + 1$ 
 $A \rightarrow B$  :  $A, B, S_A + 1, H(A, B, S_A, \text{secret},$ 
   $\text{count}) + 1$ 

```

ここで、式 $H(A, B, S_A, \text{secret}, \text{count})$ は、データ列 $A, B, S_A, \text{secret}, \text{count}$ のハッシュ値である。定数 secret は B を含め他からは秘密の数値である。

SYN-ACK のパケットにおいて、接続先 B から接続元 A に与えられる初期シーケンス番号は、通常、ランダムに定められる。ここに接続先が本来記憶しておく情報のハッシュ値を初期シーケンス番号に設定する。上記では、二番目のメッセージの $H(A, B, S_A, \text{secret}, \text{count})$ がこれである。

これにより、SYN-ACK パケット送出後、接続元は接続先情報を捨て、次の ACK パケット到着後、このハッシュ値に由来する ACK 番号 $H(A, B, S_A, \text{secret}, \text{count}) + 1$ とシーケンス番号 $S_A + 1$ やアドレス A, B などとの整合性をチェックすればよい。これが SYN クッキーという技法である。

接続元 A と接続先 B は、各々、以下のプロセス $P_A, \tilde{P}_B$ により定式化される。

```

 $P_A \stackrel{\text{def}}{=} (\text{前の場合と同じ})$ 
 $\tilde{P}_B \stackrel{\text{def}}{=} \text{new}(\text{count});$ 
  inp  $c(q_1)$ ; split  $[y_a, y_b, y_{sa}]$  is  $q_1$ 
  err{free  $q_1$ };
  free  $q_1$ ;
  match  $y_b$  is  $B$  err{free  $y_a, y_b, y_{sa}$ }; free  $y_b$ ;
  store  $y_h = \text{hash}((y_a, B, y_{sa}, \text{secret}, \text{count}))$ ;
  out  $c \langle (B, y_a, y_h, \text{succ}(y_{sa})) \rangle$ ;
  free  $y_a, y_{sa}, y_h$ ;
  inp  $c(q_2)$ ; split  $[y'_a, y'_b, y'_{sa1}, y'_{h1}]$  is  $q_2$ 
  err{free  $y_a, y_{sa}, y_h, q_2$ }; free  $q_2$ ;
  match  $y'_b$  is  $B$  err{free  $y_a, y_{sa}, y_h, y'_a, y'_b,$ 
     $y'_{sa1}, y'_{h1}$ };
  free  $y'_b$ ;
  store  $y'_h = \text{hash}((y'_a, y'_b, \text{pred}(y'_{sa1}), \text{secret},$ 
     $\text{count}))$ ;
  match  $y'_{h1}$  is  $\text{succ}(y'_h)$ 
  err{free  $y_a, y_{sa}, y_h, y'_a, y'_{sa}, y'_{h1}, y'_h$ };
  free  $y'_h$ ;  $\tilde{P}'_B$ 

```

この場合のシステム全体は、以下のようなプロセスにより表現される。

```

SYNcookieConfig  $\stackrel{\text{def}}{=} (\text{repeat } P_A \mid \text{repeat } \tilde{P}_B)$ .

```

このプロセスは次のようにコミットメント関係・簡約関係の列をもつ。

```

SYNcookieConfig
 $\equiv (\text{repeat } P_A \mid \text{repeat } \tilde{P}_B)$ 
 $\xrightarrow{\tau}$  new( $S_A$ ); new( $\text{count}$ );
  (inp  $c(p)$ ;  $\dots$ ) | repeat  $P_A$ 
  | (split  $[y_a, y_b, y_c]$  is  $q_1; \dots$ ) | repeat  $\tilde{P}_B$ 
  :  $\{\mathbf{a} \cdot \text{store}, \mathbf{b} \cdot \text{store}\}$ 

```

$$\begin{aligned}
& \gg \text{new}(S_A); \text{new}(\text{count}); \\
& (\text{inp } c(p); \dots) \mid \text{repeat } P_A \\
& \mid (\text{out } c \langle (B, A, \text{Hash}, \text{succ}(y_{sa}))) \rangle; \dots) \\
& \mid \text{repeat } \tilde{P}_B \\
& : \{\mathbf{b} \cdot 3\text{store}\} + \{\mathbf{b} \cdot -\text{store}\} \\
& + \{\mathbf{b} \cdot \text{match}\} + \{\mathbf{b} \cdot -\text{store}\} \\
& + \{\mathbf{b} \cdot \text{store}\} + \{\mathbf{b} \cdot \text{hash}\} \\
& = \{\mathbf{b} \cdot 2\text{store}, \mathbf{b} \cdot \text{match}, \mathbf{b} \cdot \text{hash}\} \\
& (\text{ただし, } \text{Hash} \equiv H(A, B, S_A, \text{secret}, \text{count})) \\
& \xrightarrow{\tau} \text{new}(S_A); \text{new}(\text{count}); \\
& (\text{split } [x'_b, x'_a, x'_{sb}, x'_{sa1}] \text{ is } p \dots) \mid \text{repeat } P_A \\
& \mid (\text{free } y_a, y_{sa}, y_h; \dots) \mid \text{repeat } \tilde{P}_B \\
& : \{\mathbf{a} \cdot \text{store}\} \\
& \gg \text{new}(S_A); \text{new}(\text{count}); \\
& (\text{out } c \langle (A, B, \text{succ}(S_A), \text{succ}(\text{Hash}))) \rangle; \dots) \\
& \mid \text{repeat } P_A \\
& \mid (\text{free } y_a, y_{sa}, y_h; \dots) \mid \text{repeat } \tilde{P}_B \\
& : \{\mathbf{a} \cdot 4\text{store}\} + \{\mathbf{a} \cdot -\text{store}\} \\
& + \{\mathbf{a} \cdot 3\text{match}\} - \{\mathbf{a} \cdot 3\text{store}\} \\
& = \{\mathbf{a} \cdot 3\text{match}\} \\
& \xrightarrow{\tau} \text{new}(S_A); \text{new}(\text{count}); \\
& P'_A \mid \text{repeat } P_A \\
& \mid \text{split } [y'_a, y'_b, y'_{sa1}, y'_{h1}] \text{ is } q_2 \dots \\
& \mid \text{repeat } \tilde{P}_B \\
& : \{\mathbf{b} \cdot -3\text{store}\} + \{\mathbf{b} \cdot \text{store}\} \\
& = \{\mathbf{b} \cdot -2\text{store}\} \\
& \gg \text{new}(S_A); P'_A \mid \text{repeat } P_A \mid \tilde{P}'_B \mid \text{repeat } \tilde{P}_B \\
& : \{\mathbf{b} \cdot 4\text{store}\} + \{\mathbf{b} \cdot -\text{store}\} \\
& + \{\mathbf{b} \cdot \text{match}\} + \{\mathbf{b} \cdot -\text{store}\} \\
& + \{\mathbf{b} \cdot \text{store}\} + \{\mathbf{b} \cdot \text{hash}\} \\
& + \{\mathbf{b} \cdot \text{match}\} \\
& = \{\mathbf{b} \cdot 3\text{store}, \mathbf{b} \cdot \text{hash}, \mathbf{b} \cdot \text{match}\}
\end{aligned}$$

コストは通算すると以下ようになる。

SYNcookieConfig

$$\begin{aligned}
& \rightarrow \text{new}(S_A); \text{new}(\text{cookie}); \\
& (P'_A \mid \text{repeat } P_A \mid \tilde{P}'_B \mid \text{repeat } \tilde{P}_B) \\
& : \{\mathbf{a} \cdot 2\text{store}, \mathbf{a} \cdot 3\text{match}, \mathbf{b} \cdot 3\text{store}, \\
& \quad \mathbf{b} \cdot 2\text{match}, \mathbf{b} \cdot 2\text{hash}\}
\end{aligned}$$

SYN クッキーのもとで、SYN あふれ攻撃が行なわれると以下ようになる。

DefendedConfig

$$\stackrel{\text{def}}{=} (\text{repeat } P_I \mid \text{repeat } P_B \mid \text{repeat } P_N).$$

DefendedConfig

$$\equiv (\text{repeat } P_I \mid \text{repeat } \tilde{P}_B \mid \text{repeat } P_N)$$

$$\xrightarrow{\tau} \text{new}(S_B);$$

(stop $\mid$ repeat P_A

$\mid$ (split $[y_a, y_b, y_c]$ is $q_1; \dots$)

$\mid$ repeat $\tilde{P}_B$

$\mid$ inp $c(r); \text{stop} \mid \text{repeat } P_N)$

: $\{\mathbf{b} \cdot \text{store}\}$

$$\gg \text{new}(S_B);$$

(stop $\mid$ repeat P_A

$\mid$ out $c \langle (B, A, S_B, \text{succ}(S_A))) \rangle; \dots$

$\mid$ repeat $\tilde{P}_B$

$\mid$ inp $c(r); \text{stop} \mid \text{repeat } P_N)$

: $\{\mathbf{b} \cdot 3\text{store}\} + \{\mathbf{b} \cdot -\text{store}\}$

+ $\{\mathbf{b} \cdot \text{match}\} + \{\mathbf{b} \cdot -\text{store}\}$

+ $\{\mathbf{b} \cdot \text{store}\} + \{\mathbf{b} \cdot \text{hash}\}$

= $\{\mathbf{b} \cdot 2\text{store}, \mathbf{b} \cdot \text{match}, \mathbf{b} \cdot \text{hash}\}$

$$\gg \text{new}(S_B);$$

(stop $\mid$ repeat $P_A \mid$ inp $c(q_2); \dots$

$\mid$ repeat $\tilde{P}_B \mid \text{stop} \mid \text{repeat } P_N)$

: $\{\mathbf{b} \cdot -3\text{store}, \mathbf{n} \cdot \text{store}\}$

この場合、コストは通算して以下ようになる。

DefendedConfig

$$\rightarrow \text{new}(S_B);$$

(stop $\mid$ repeat $P_A \mid$ inp $c(q_2); \dots$

$\mid$ repeat $\tilde{P}_B \mid \text{stop} \mid \text{repeat } P_N) \text{new}(S_B);$

: $\{\mathbf{b} \cdot \text{hash}, \mathbf{b} \cdot \text{match}, \mathbf{n} \cdot \text{store}\}.$

この結果より次のことがわかる。

- SYN クッキーを用いる場合は、用いない場合と比較して、ハッシュの計算コストがサーバーにおいて余計に必要となる。
- しかしながら、SYN クッキーを用いる場合にサーバー側において必要とされた記憶コストが、SYN クッキーを用いない場合において必要でなくなった。

上記であげた例では、通常のクライアント A や攻撃者 I を与え、各々に対するサーバー側の計算コストを比較して議論した。一般に、プロトコルの安全

性検証では、あらゆる通信パターンに対して、サーバーが安全であることを示さなければならない。上記の SYN クッキーの機構を導入したサーバー $\tilde{P}_B$ の場合は、 $\tilde{P}_B$ から始まるすべての遷移を考え、そのコストを検討すればよい。図 1 に遷移図を与える。このグラフは、節はプロセスを表し、有向辺 (矢印) は、コミットメント関係や簡約関係による遷移を表す。有向辺の右側に規則名を付記し、左側に発生するコスト (ただし計算機名は省略) を付記している。この $\tilde{P}_B$ が、受信待ち状態となるのは、図の節 (a) である。ここまでに必要なコストは、 $match + hash$ であり、記憶コストがかからないことがわかる。サーバー自体は、 $repeat \tilde{P}_B$ により表され、プロセス $\tilde{P}_B$ は、セッションの一つに対応する。通信待ち状態が他の状態と比べて長時間継続することと、この時点で、記憶コストが 0 であることから、DoS 耐性が高いことがわかる。

4 おわりに

本論文では、プロトコルにおけるサービス不能攻撃に対する耐性を記述・解析するための形式的体系として Spice 計算を提唱した。この Spice 計算は、pi 計算・spi 計算を基にしたものとなっている。Spice 計算では、複数の計算機の構成状態を記述する機構として型を導入し、プロトコルの処理の進行において計算機毎に発生するコストを定式化することを可能にした。理論的性質として、主部簡約定理と記憶コスト健全性を示した。また、本体系におけるプロトコルの記述の例として、SYN あふれ攻撃とその防御法の一つである SYN クッキーを記述した。

ただ、前章で議論した、SYN あふれ攻撃と SYN クッキーによる防御法の解析は、すべての通信パターンに関する議論を行なったものではない。ここで議論している TCP の 3 ウェイハンドシェイクが、本当にサービス不能攻撃耐性をもつことを保証するためには、攻撃者からのあらゆる攻撃パターンに対して、安全であることを示さなければならない。

論文で与えた例では、サービス不能攻撃の攻撃パターンは、人間が与えたものであった。どのような攻撃パターンがありうるのかということ (半) 自動で

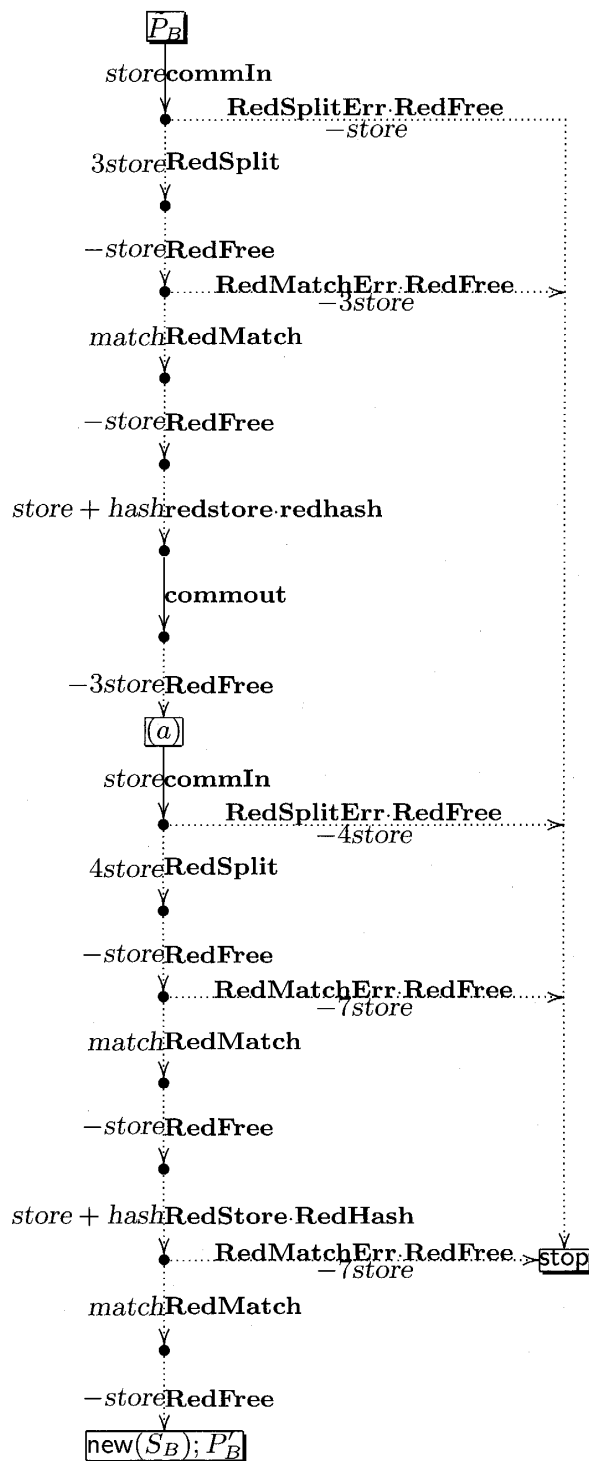


図 1 $\tilde{P}_B$ の遷移図

行なう手法を確立することが重要である。これは今後の課題の一つである。

また、バッファあふれ攻撃 [18] に対する安全性に関しては、静的解析の研究も進んでいる [14][15]。この手法は、手続型言語で記述されたソースコードのデータフロー解析・コントロールフロー解析を用いて、プログラム中で使用されている各バッファの利用長を見

積み, バッファあふれの可能性を検出するというものである。バッファあふれ攻撃に対する安全性の解析では, 逐次型のプログラムにおいて, 動的に使用されるバッファ使用量をいかに静的に詳細に見積るかということが課題になる。一方, 本論文があつかったサービス不能攻撃耐性においては, 攻撃者のプロセスと被攻撃者のプロセスとの消費コストのバランスが解析の対象である。被攻撃者プロセスにおけるコスト消費がさほど大したものではなくとも, それと対をなす攻撃者のプロセスのコスト消費が軽いものであれば, サービス不能攻撃耐性の観点からは問題となる。そのため, 攻撃者と被攻撃者とを包括して捉えることができる形式的枠組みでなくてはならない。また, プロトコルで消費される計算コストは, 通信のデータ量が一定であるならば, そのデータの中身にはあまり依存しない。このようなことから, バッファあふれ攻撃の安全性解析の研究とは, 異なるアプローチとなった。

最近, 住井ら[17]により, アリス・ボブ記法(略式記法)から, spi 計算への変換が提案されている。この変換を基にして, メドーズの拡張アリス・ボブ記法の体系から, Spice 計算への変換を与えられると考えられる。これも将来の興味深い研究方向である。

参考文献

- [1] Abadi, M. and Gordon, A. D.: A Calculus for Cryptographic Protocols: The Spi Calculus, in *Fourth ACM Conference on Computer and Communication Security*, ACM Press, 1997, pp. 36–47.
- [2] Abadi, M. and Gordon, A. D.: Reasoning about cryptographic protocols in the spi calculus, in *CONCUR'97: Concurrency Theory*, Vol. 1243, Springer-Verlag, Berlin Germany, 1997, pp. 59–73.
- [3] Aura, T. and Nikander, P.: Stateless connections, in *International Conference on Information and Communications Security ICICS'97*, Lecture Notes in Computer Science, Vol. 1334, Springer-Verlag, 1997, pp. 87–97.
- [4] Aura, T., Nikander, P. and Leiwo, J.: DOS-Resistant Authentication with Client Puzzles, in *Security Protocols, 8th International Workshop*, Lecture Notes in Computer Science, Vol. 2133, Springer-Verlag, 2001, pp. 170–177.
- [5] Gordon, A. D. and Jeffrey, A.: Authenticity by typing for security protocols, in *14th IEEE Computer Security Foundations Workshop*, 2001, pp. 145–159.
- [6] Gordon, A. D. and Jeffrey, A.: Typing correspondence assertions for communication protocols, *Electronic Notes in Theoretical Computer Science*, Vol. 45, No. 17(2001), pp. 1–22.
- [7] Gordon, A. D. and Jeffrey, A.: Types and effects for asymmetric cryptographic protocols, in *15th IEEE Computer Security Foundations Workshop*, 2002, pp. 77–91.
- [8] Meadows, C.: A formal framework and evaluation method for network denial of service, in *Proceeding of the 12th IEEE Computer Security Foundations Workshop*, 1999, pp. 4–13.
- [9] Meadows, C.: A Cost-Based Framework for Analysis of Denial of Service Networks, *Journal of Computer Security*, Vol. 9, No. 1/2(2001), pp. 143–164.
- [10] Millen, J. K.: A Resource Allocation Model for Denial of Service Protection, *Journal of Computer Security*, Vol. 2, No. 2/3(1993), pp. 89–106.
- [11] Milner, R., Parrow, J. and Walker, D.: A calculus of mobile processes, Part I and Part II, *Information and Computation*, Vol. 100, No. 1(1992), pp. 1–77.
- [12] Schuba, C. L., Krsul, I. V., Kuhn, M. G., Spafford, E. H., Sundaram, A. and Zamboni, D.: Analysis of a Denial of Service Attack on TCP, in *Proceedings of the 1997 IEEE Symposium on Security and Privacy*, IEEE Computer Society, IEEE Computer Society Press, May 1997, pp. 208–223.
- [13] Tomioka, D., Nishizaki, S. and Ikeda, R.: A Cost Estimation Calculus for Analyzing the Resistance to Denial-of-Service Attack, in *Software Security – Theories and Systems*, Lecture Notes in Computer Science, Vol. 3233, Springer-Verlag, 2004, pp. 25–44.
- [14] Wagner, D., Foster, J., Brewer, E. and Aiken, A.: A first step towards automated detection of buffer overrun vulnerabilities, in *Network and Distributed System Security Symposium, NDSS 2000*, 1999, pp. 3–17.
- [15] Weber, M., Shah, V. and Ren, C.: A Case Study in Detecting Software Security Vulnerabilities Using Constraint Optimization, in *First IEEE International Workshop on Source Code Analysis and Manipulation, SCAM 2001*, 2001, pp. 3–13.
- [16] 住井英二郎: 暗号化通信の spi 計算による形式的検証, *コンピュータソフトウェア*, Vol. 20, No. 6(2003), pp. 75–84.
- [17] 住井英二郎, 立沢秀晃, 米澤明憲: セキュリティプロトコルの略式表現から spi 計算への変換, *情報処理学会論文誌*, Vol. 45, No. SIG12 (2004), pp. 1–10.
- [18] 脇田建: バッファ溢れ攻撃とその防御, *コンピュータソフトウェア*, Vol. 19, No. 1(2002), pp. 49–63.